Supplemental: Formalized Lean 4 Proofs Contexts and Pseudocontexts

Mirko Navara Karl Svozil

June 2, 2026

Overview

This document gives the complete Lean 4 source for the formal verification accompanying the manuscript *Contexts and pseudocontexts*. The Lean development formalizes the finite coordinate-operator core of the paper: rank-one projectors, finite structure operators, the complex phase construction, ray-disjointness, the revised genuineness certificate, and the basic trace identities for rank-one decompositions.

The formalization is intentionally scoped to the algebraic spine of the article. It does not attempt to formalize the full projection-lattice interface, Gleason-theorem infrastructure, Schur–Horn theory, or the topological genericity arguments used elsewhere in the manuscript. Instead, it supplies a machine-checked certificate for the explicit finite-dimensional operator calculations that are most directly amenable to Lean verification.

The source has been checked with Lean 4 `v4.30.0` and mathlib `v4.30.0`. In the archived workspace used for this supplement, the verification command is

```
powershell.exe -ExecutionPolicy Bypass -File .\verify.ps1.
```

The Lean files contain no placeholders such as `sorry` or `admit`.

The formalization consists of a single main Lean file:

- Section 2 (`Pseudocontexts/Basic.lean`) defines coordinate rank-one operators, structure operators, the root-of-unity phase construction, same-ray predicates, the revised plane-genuineness certificate, and trace identities.

Contents

1 Project Configuration Files

lean-toolchain

```
leanprover/lean4:v4.30.0
```

lakefile.lean

```
import Lake
open Lake DSL

package Pseudocontexts where

require batteries from "C:/Users/a/Documents/Codex/2026-05-30/can-you-help-me-with-lean/.lake/packages/batteries"
require Qq from "C:/Users/a/Documents/Codex/2026-05-30/can-you-help-me-with-lean/.lake/packages/Qq"
require aesop from "C:/Users/a/Documents/Codex/2026-05-30/can-you-help-me-with-lean/.lake/packages/aesop"
require Cli from "C:/Users/a/Documents/Codex/2026-05-30/can-you-help-me-with-lean/.lake/packages/Cli"
require proofwidgets from "C:/Users/a/Documents/Codex/2026-05-30/can-you-help-me-with-lean/.lake/packages/proofwidgets"
require importGraph from "C:/Users/a/Documents/Codex/2026-05-30/can-you-help-me-with-lean/.lake/packages/importGraph"
require LeanSearchClient from "C:/Users/a/Documents/Codex/2026-05-30/can-you-help-me-with-lean/.lake/packages/LeanSearchClient"
require plausible from "C:/Users/a/Documents/Codex/2026-05-30/can-you-help-me-with-lean/.lake/packages/plausible"
require mathlib from "C:/Users/a/Documents/Codex/2026-05-30/can-you-help-me-with-lean/.lake/packages/mathlib"

@[default_target]
lean_lib Pseudocontexts where
```

Pseudocontexts.lean

```
import Pseudocontexts.Basic
```

verify.ps1

```
$ErrorActionPreference = "Stop"

$cache = "C:\Users\a\Documents\Codex\2026-05-30\can-you-help-me-with-lean\.lake\packages"
```

```
$lean = "C:\Users\a\.elan\toolchains\leanprover--lean4---v4.30.0\bin\lean.exe"
$file = Join-Path $PSScriptRoot "Pseudocontexts\Basic.lean"

$paths = Get-ChildItem -Directory $cache |
  ForEach-Object { Join-Path $_.FullName ".lake\build\lib\lean" } |
  Where-Object { Test-Path $_ }

$env:LEAN_PATH = ($paths -join ";")
& $lean $file
```

2 Formalized Pseudocontext Algebra

Pseudocontexts/Basic.lean

```
import Mathlib.Data.Complex.Basic
import Mathlib.Data.Fintype.Basic
import Mathlib.Algebra.BigOperators.Group.Finset.Basic
import Mathlib.Algebra.BigOperators.Ring.Finset
import Mathlib.Data.Matrix.Mul
import Mathlib.Tactic.FinCases
import Mathlib.Tactic.Ring

open scoped BigOperators

namespace Pseudocontexts

set_option linter.unusedSectionVars false
set_option linter.unusedSimpArgs false
set_option linter.unnecessarySimpa false

/-!
# Formal algebra for pseudocontexts

This file formalizes the operator-level algebra behind the paper's
pseudocontext proofs.  We work with finite coordinate vectors over ℂ``
and rank-one matrices `|⟩⟨vv|` represented as functions.
-/

abbrev CMatrix (n : Type) := n → n →ℂ

variable ι{ n : Type} [Fintype ι] [DecidableEq ι]
  [Fintype n] [DecidableEq n]

/-- The rank-one operator `|⟩⟨vv|`, in coordinates. -/
def rankOne (v : n →ℂ ) : CMatrix n :=
  fun p q => v p * star (v q)

/-- The structure operator of a finite tuple of vectors. -/
def structureOp (a : ι→  n →ℂ ) : CMatrix n :=
  fun p q => ∑ i : ι, rankOne (a i) p q

/-- Equality of rank-one sums is exactly equality of all matrix entries. -/
theorem structure_ext {a b : ι→  n →ℂ }
    (h : ∀ p q, ∑( i : ι, rankOne (a i) p q)
      = ∑( i : ι, rankOne (b i) p q)) :
    structureOp a = structureOp b := by
  funext p q
  exact h p q

```

```
/-- Two vectors span the same complex ray when one is a nonzero scalar
multiple of the other. -/
def SameRay (u v : n →ℂ ) : Prop :=∃
   z : ℂ, z ≠ 0 ∧∀  p, v p = z * u p

namespace PhaseConstruction

/-- The vectors `a_j = c ₁e + s ω^j ₂e` in ℂ`². -/
def avec (c s ω : ℂ) (j : ℕ) : Fin 2 →ℂ
  | 0 => c
  | 1 => s * ω ^ j

/-- The phase-shifted vectors `b_j = c ₁e + s ηω ^j ₂e` in ℂ`². -/
def bvec (c s ηω  : ℂ) (j : ℕ) : Fin 2 →ℂ
  | 0 => c
  | 1 => s * η( * ω ^ j)

/-- Coordinate inner product on the two-dimensional support used by the
phase construction. -/
def inner2 (u v : Fin 2 →ℂ ) : ℂ :=∑
   p : Fin 2, star (u p) * v p

/-- A tuple is pairwise nonorthogonal when no two distinct entries are
orthogonal.  This is the formal version of the α` ≠π /4` branch in the
paper's phase construction. -/
def PairwiseNonorthogonal {k : ℕ} (a : Fin k → Fin 2 →ℂ ) : Prop :=∀
   j l : Fin k, j ≠ l → inner2 (a j) (a l) ≠ 0

/-- In the fixed two-dimensional support of the phase construction, a
context has exactly two rays.  This records the cardinality obstruction
used in the revised paper. -/
def PlaneContextCardinality (k : ℕ) : Prop :=
  k = 2

/-- A light-weight certificate that a tuple is not a context in the
two-dimensional support: either its cardinality is not two, or it is
pairwise nonorthogonal. -/
def NotContextInPlane {k : ℕ} (a : Fin k → Fin 2 →ℂ ) : Prop :=
  ¬ PlaneContextCardinality k ∨ PairwiseNonorthogonal a

/-- The revised genuineness certificate for the complex phase construction:
both tuples fail to be contexts in their common two-dimensional support. -/
def PhaseGenuineInPlane (c s ηω  : ℂ) (k : ℕ) : Prop :=
  NotContextInPlane (fun j : Fin k => avec c s ω (j : ℕ)) ∧
  NotContextInPlane (fun j : Fin k => bvec c s ηω  (j : ℕ))

theorem not_planeContextCardinality_of_three {k : ℕ} (hk : 3 ≤ k) :
    ¬ PlaneContextCardinality k := by
  intro h
  unfold PlaneContextCardinality at h
```

```
  have hbad : 3 ≤ 2 := by simpa [h] using hk
  exact (by decide : ¬ 3 ≤ 2) hbad

/-- This is the Lean-level counterpart of the revised paper statement
``genuine whenever `k ≥ 3` or α` ≠π /4`''.  The second branch is phrased
as the pairwise-nonorthogonality certificate produced by the
inner-product computation in the paper. -/
theorem phase_genuine_in_plane_of_three_or_pairwise {k : ℕ} {c s ηω  : ℂ}
    (h : 3 ≤ k ∨
      (PairwiseNonorthogonal (fun j : Fin k => avec c s ω (j : ℕ)) ∧
       PairwiseNonorthogonal (fun j : Fin k => bvec c s ηω  (j : ℕ)))) :
    PhaseGenuineInPlane c s ηω  k := by
  rcases h with hk | hpair
  · constructor
    · exact Or.inl (not_planeContextCardinality_of_three hk)
    · exact Or.inl (not_planeContextCardinality_of_three hk)
  · exact ⟨Or.inr hpair.1, Or.inr hpair⟩.2

theorem sum_rankOne_avec_apply_zero_zero (c s ω : ℂ) (k : ℕ) :∑
    ( j : Fin k, rankOne (avec c s ω (j : ℕ)) 0 0)
      = k • (c * star c) := by
  simp [rankOne, avec]

theorem sum_rankOne_bvec_apply_zero_zero (c s ηω  : ℂ) (k : ℕ) :∑
    ( j : Fin k, rankOne (bvec c s ηω  (j : ℕ)) 0 0)
      = k • (c * star c) := by
  simp [rankOne, bvec]

theorem sum_rankOne_avec_apply_zero_one {c s ω : ℂ} {k : ℕ}
    (ωhstar : ∑( j : Fin k, star ω( ^ (j : ℕ))) = 0) :∑
    ( j : Fin k, rankOne (avec c s ω (j : ℕ)) 0 1) = 0 := by
  calc∑
    ( j : Fin k, rankOne (avec c s ω (j : ℕ)) 0 1)
        = ∑ j : Fin k, c * star s * star ω( ^ (j : ℕ)) := by
          simp [rankOne, avec, mul_assoc]
    _ = c * star s * ∑( j : Fin k, star ω( ^ (j : ℕ))) := by
          rw ←[ Finset.mul_sum]
    _ = 0 := by rw [ωhstar]; simp

theorem sum_rankOne_bvec_apply_zero_one {c s ηω  : ℂ} {k : ℕ}
    (ωhstar : ∑( j : Fin k, star ω( ^ (j : ℕ))) = 0) :∑
    ( j : Fin k, rankOne (bvec c s ηω  (j : ℕ)) 0 1) = 0 := by
  calc∑
    ( j : Fin k, rankOne (bvec c s ηω  (j : ℕ)) 0 1)
        = ∑ j : Fin k, c * star s * star η * star ω( ^ (j : ℕ)) := by
          simp [rankOne, bvec, mul_assoc]
    _ = c * star s * star η * ∑( j : Fin k, star ω( ^ (j : ℕ))) := by
          rw ←[ Finset.mul_sum]
    _ = 0 := by rw [ωhstar]; simp

```

```
theorem sum_rankOne_avec_apply_one_zero {c s ω : ℂ} {k : ℕ}
    (ωh : ∑( j : Fin k, ω ^ (j : ℕ)) = 0) :∑
    ( j : Fin k, rankOne (avec c s ω (j : ℕ)) 1 0) = 0 := by
  calc∑
    ( j : Fin k, rankOne (avec c s ω (j : ℕ)) 1 0)
        = ∑ j : Fin k, s * star c * ω ^ (j : ℕ) := by
          simp [rankOne, avec, mul_assoc, mul_comm, mul_left_comm]
    _ = s * star c * ∑( j : Fin k, ω ^ (j : ℕ)) := by
          rw ←[ Finset.mul_sum]
    _ = 0 := by rw [ωh]; simp

theorem sum_rankOne_bvec_apply_one_zero {c s ηω  : ℂ} {k : ℕ}
    (ωh : ∑( j : Fin k, ω ^ (j : ℕ)) = 0) :∑
    ( j : Fin k, rankOne (bvec c s ηω  (j : ℕ)) 1 0) = 0 := by
  calc∑
    ( j : Fin k, rankOne (bvec c s ηω  (j : ℕ)) 1 0)
        = ∑ j : Fin k, s * star c * η * ω ^ (j : ℕ) := by
          simp [rankOne, bvec, mul_assoc, mul_comm, mul_left_comm]
    _ = s * star c * η * ∑( j : Fin k, ω ^ (j : ℕ)) := by
          rw ←[ Finset.mul_sum]
    _ = 0 := by rw [ωh]; simp

theorem sum_rankOne_avec_apply_one_one {c s ω : ℂ} {k : ℕ}
    (ωhnorm : ∀ j : Fin k, ω ^ (j : ℕ) * star ω( ^ (j : ℕ)) = 1) :∑
    ( j : Fin k, rankOne (avec c s ω (j : ℕ)) 1 1)
      = k • (s * star s) := by
  calc∑
    ( j : Fin k, rankOne (avec c s ω (j : ℕ)) 1 1)
        = ∑ j : Fin k, s * star s *ω
            ( ^ (j : ℕ) * star ω( ^ (j : ℕ))) := by
          simp [rankOne, avec, mul_assoc, mul_comm, mul_left_comm]
    _ = ∑ _j : Fin k, s * star s := by
          refine Finset.sum_congr rfl ?_
          intro j _
          calc
            s * star s * ω( ^ (j : ℕ) * star ω( ^ (j : ℕ)))
                = s * star s * 1 := by rw [ωhnorm j]
            _ = s * star s := by ring
    _ = k • (s * star s) := by simp

theorem sum_rankOne_bvec_apply_one_one {c s ηω  : ℂ} {k : ℕ}
    (ηhnorm : η * star η = 1)
    (ωhnorm : ∀ j : Fin k, ω ^ (j : ℕ) * star ω( ^ (j : ℕ)) = 1) :∑
    ( j : Fin k, rankOne (bvec c s ηω  (j : ℕ)) 1 1)
      = k • (s * star s) := by
  calc∑
    ( j : Fin k, rankOne (bvec c s ηω  (j : ℕ)) 1 1)
        = ∑ j : Fin k, s * star s * η( * star η) *ω
            ( ^ (j : ℕ) * star ω( ^ (j : ℕ))) := by
          simp [rankOne, bvec, mul_assoc, mul_comm, mul_left_comm]
```

```
    _ = ∑ _j : Fin k, s * star s := by
          refine Finset.sum_congr rfl ?_
          intro j _
          calc
            s * star s * η( * star η) *ω
                ( ^ (j : ℕ) * star ω( ^ (j : ℕ)))
                = s * star s * 1 * 1 := by rw [ηhnorm, ωhnorm j]
            _ = s * star s := by ring
    _ = k • (s * star s) := by simp

/-- Algebraic core of the complex-phase construction: a phase-shifted
root-of-unity tuple gives the same structure operator. -/
theorem phase_structure_eq {k : ℕ} {c s ηω  : ℂ}
    (ωh : ∑( j : Fin k, ω ^ (j : ℕ)) = 0)
    (ωhstar : ∑( j : Fin k, star ω( ^ (j : ℕ))) = 0)
    (ηhnorm : η * star η = 1)
    (ωhnorm : ∀ j : Fin k, ω ^ (j : ℕ) * star ω( ^ (j : ℕ)) = 1) :
    structureOp (fun j : Fin k => avec c s ω (j : ℕ))
      = structureOp (fun j : Fin k => bvec c s ηω  (j : ℕ)) := by
  funext p q
  fin_cases p <;> fin_cases q
  · simp [structureOp, sum_rankOne_avec_apply_zero_zero,
      sum_rankOne_bvec_apply_zero_zero]
  · simp [structureOp, sum_rankOne_avec_apply_zero_one ωhstar,
      sum_rankOne_bvec_apply_zero_one ωhstar]
  · simp [structureOp, sum_rankOne_avec_apply_one_zero ωh,
      sum_rankOne_bvec_apply_one_zero ωh]
  · simp [structureOp, sum_rankOne_avec_apply_one_one ωhnorm,
      sum_rankOne_bvec_apply_one_one ηhnorm ωhnorm]

/-- If two `a`-vectors are on the same ray, the corresponding powers
of ω`` agree. -/
theorem sameRay_avec_imp_pow_eq {c s ω : ℂ} {j l : ℕ}
    (hc : c ≠ 0) (hs : s ≠ 0)
    (h : SameRay (avec c s ω j) (avec c s ω l)) :ω
     ^ l = ω ^ j := by
  rcases h with ⟨z, _hz, ⟩hzv
  have hz : z = 1 := by
    have h0 := hzv 0
    have h0' : (1 : ℂ) * c = z * c := by simpa [avec] using h0
    exact (₀mul_right_cancel hc h0').symm
  have h1 := hzv 1
  have h1' : s * ω ^ l = s * ω ^ j := by simpa [avec, hz] using h1
  exact ₀mul_left_cancel hs h1'

/-- If two phase-shifted `b`-vectors are on the same ray, the
corresponding powers of ω`` agree. -/
theorem sameRay_bvec_imp_pow_eq {c s ηω  : ℂ} {j l : ℕ}
    (hc : c ≠ 0) (hs : s ≠ 0) (ηh : η≠  0)
    (h : SameRay (bvec c s ηω  j) (bvec c s ηω  l)) :ω
```

```
    ^ l = ω ^ j := by
  rcases h with ⟨z, _hz, ⟩hzv
  have hz : z = 1 := by
    have h0 := hzv 0
    have h0' : (1 : ℂ) * c = z * c := by simpa [bvec] using h0
    exact (₀mul_right_cancel hc h0').symm
  have h1 := hzv 1
  have h1' : s * η * ω ^ l = s * η * ω ^ j := by
    simpa [bvec, hz, mul_assoc] using h1
  exact ₀mul_left_cancel (mul_ne_zero hs ηh) h1'

/-- If an `a`-vector and a phase-shifted `b`-vector are on the same
ray, then the phase relation η` ω^l = ω^j` holds. -/
theorem sameRay_avec_bvec_imp_phase_eq {c s ηω  : ℂ} {j l : ℕ}
    (hc : c ≠ 0) (hs : s ≠ 0)
    (h : SameRay (avec c s ω j) (bvec c s ηω  l)) :η
     * ω ^ l = ω ^ j := by
  rcases h with ⟨z, _hz, ⟩hzv
  have hz : z = 1 := by
    have h0 := hzv 0
    have h0' : (1 : ℂ) * c = z * c := by simpa [avec, bvec] using h0
    exact (₀mul_right_cancel hc h0').symm
  have h1 := hzv 1
  have h1' : s * η( * ω ^ l) = s * ω ^ j := by
    simpa [avec, bvec, hz, mul_assoc] using h1
  exact ₀mul_left_cancel hs h1'

/-- A compact formal pseudocontext statement for the complex-phase
construction.  The hypotheses say exactly that the phases cancel in
the off-diagonal entries, have unit modulus on the diagonal, and do not
create ray collisions. -/
theorem complex_phase_pseudocontext {k : ℕ} {c s ηω  : ℂ}
    (hc : c ≠ 0) (hs : s ≠ 0)
    (ωh : ∑( j : Fin k, ω ^ (j : ℕ)) = 0)
    (ωhstar : ∑( j : Fin k, star ω( ^ (j : ℕ))) = 0)
    (ηhnorm : η * star η = 1)
    (ωhnorm : ∀ j : Fin k, ω ^ (j : ℕ) * star ω( ^ (j : ℕ)) = 1)
    (ha_distinct : ∀ j l : Fin k, ω ^ (j : ℕ) = ω ^ (l : ℕ) → j = l)
    (hcross : ∀ j l : Fin k, η * ω ^ (l : ℕ) ≠ω  ^ (j : ℕ)) :
    structureOp (fun j : Fin k => avec c s ω (j : ℕ))
        = structureOp (fun j : Fin k => bvec c s ηω  (j : ℕ))∧∀
       ( j l : Fin k,
          SameRay (avec c s ω (j : ℕ)) (avec c s ω (l : ℕ)) → j = l)∧∀
       ( j l : Fin k,
          SameRay (bvec c s ηω  (j : ℕ)) (bvec c s ηω  (l : ℕ)) → j = l)∧∀
       ( j l : Fin k,
          ¬ SameRay (avec c s ω (j : ℕ)) (bvec c s ηω  (l : ℕ))) := by
  refine ⟨phase_structure_eq ωh ωhstar ηhnorm ωhnorm, ?_, ?_, ?⟩_
  · intro j l h
    apply ha_distinct
```

```
    exact (sameRay_avec_imp_pow_eq hc hs h).symm
  · intro j l h
    apply ha_distinct
    have ηh : η≠  0 := fun ηh0 => by
      have : (0 : ℂ) = 1 := by
        simpa [ηh0] using ηhnorm
      exact zero_ne_one this
    exact (sameRay_bvec_imp_pow_eq hc hs ηh h).symm
  · intro j l h
    exact hcross j l (sameRay_avec_bvec_imp_phase_eq hc hs h)

/-- The complex phase construction together with the revised genuineness
certificate: the common-operator and ray-disjointness conclusions are the
same as in `complex_phase_pseudocontext`, and genuineness is certified by
either `k ≥ 3` or pairwise nonorthogonality of both tuples. -/
theorem complex_phase_pseudocontext_with_genuineness {k : ℕ} {c s ηω  : ℂ}
    (hc : c ≠ 0) (hs : s ≠ 0)
    (ωh : ∑( j : Fin k, ω ^ (j : ℕ)) = 0)
    (ωhstar : ∑( j : Fin k, star ω( ^ (j : ℕ))) = 0)
    (ηhnorm : η * star η = 1)
    (ωhnorm : ∀ j : Fin k, ω ^ (j : ℕ) * star ω( ^ (j : ℕ)) = 1)
    (ha_distinct : ∀ j l : Fin k, ω ^ (j : ℕ) = ω ^ (l : ℕ) → j = l)
    (hcross : ∀ j l : Fin k, η * ω ^ (l : ℕ) ≠ω  ^ (j : ℕ))
    (hgenuine : 3 ≤ k ∨
      (PairwiseNonorthogonal (fun j : Fin k => avec c s ω (j : ℕ)) ∧
       PairwiseNonorthogonal (fun j : Fin k => bvec c s ηω  (j : ℕ)))) :
    (structureOp (fun j : Fin k => avec c s ω (j : ℕ))
        = structureOp (fun j : Fin k => bvec c s ηω  (j : ℕ))∧∀
       ( j l : Fin k,
          SameRay (avec c s ω (j : ℕ)) (avec c s ω (l : ℕ)) → j = l)∧∀
       ( j l : Fin k,
          SameRay (bvec c s ηω  (j : ℕ)) (bvec c s ηω  (l : ℕ)) → j = l)∧∀
       ( j l : Fin k,
          ¬ SameRay (avec c s ω (j : ℕ)) (bvec c s ηω  (l : ℕ)))) ∧
      PhaseGenuineInPlane c s ηω  k := by
  exact ⟨complex_phase_pseudocontext hc hs ωh ωhstar ηhnorm ωhnorm
      ha_distinct hcross,
    phase_genuine_in_plane_of_three_or_pairwise ⟩hgenuine

end PhaseConstruction

namespace TraceIdentities

variable ι{ n : Type} [Fintype ι] [DecidableEq ι]
  [Fintype n] [DecidableEq n]

/-- Coordinate inner product, conjugate-linear in the first variable. -/
def inner (u v : n →ℂ ) : ℂ :=∑
   p : n, star (u p) * v p

```

```
/-- Matrix product for coordinate matrices. -/
def mmul (A B : CMatrix n) : CMatrix n :=
  fun p q => ∑ r : n, A p r * B r q

/-- Coordinate trace. -/
def trace (A : CMatrix n) : ℂ :=∑
   p : n, A p p

theorem trace_rankOne (v : n →ℂ ) :
    trace (rankOne v) = inner v v := by
  simp [trace, inner, rankOne, mul_comm]

theorem trace_structure (a : ι→  n →ℂ )
    (hunit : ∀ i, inner (a i) (a i) = 1) :
    trace (structureOp a) = Fintype.card ι := by
  calc
    trace (structureOp a)
        = ∑ p : n, ∑ i : ι, rankOne (a i) p p := by
          rfl
    _ = ∑ i : ι, trace (rankOne (a i)) := by
          rw [Finset.sum_comm]
          rfl
    _ = ∑ i : ι, inner (a i) (a i) := by
          simp [trace_rankOne]
    _ = ∑ _i : ι, (1 : ℂ) := by
          exact Finset.sum_congr rfl (by intro i _; simp [hunit i])
    _ = Fintype.card ι := by
          simp

theorem rankOne_mul_rankOne_trace (u v : n →ℂ ) :
    trace (mmul (rankOne u) (rankOne v)) = inner v u * inner u v := by
  calc
    trace (mmul (rankOne u) (rankOne v))
        = ∑ p : n, ∑ r : n,
            (u p * star (u r)) * (v r * star (v p)) := by
          rfl
    _ = ∑ p : n, (u p * star (v p)) *∑
            ( r : n, star (u r) * v r) := by
          simp_rw [Finset.mul_sum]
          congr
          ext p
          apply Finset.sum_congr rfl
          intro r _
          ring
    _ = ∑( p : n, u p * star (v p)) * inner u v := by
          rw [Finset.sum_mul]
          rfl
    _ = inner v u * inner u v := by
          congr
          ext p
```

```
          ring

theorem sum_rankOne_mul_rankOne_trace (a : ι→  n →ℂ ) :∑
    ( i : ι, ∑ j : ι,
        trace (mmul (rankOne (a i)) (rankOne (a j))))
      = ∑ i : ι, ∑ j : ι, inner (a j) (a i) * inner (a i) (a j) := by
  simp [rankOne_mul_rankOne_trace]

end TraceIdentities

end Pseudocontexts
```